

BUT Informatique - R4.12

Automates et Langages

Bases de la Théorie des langages et des automates

J. Christian Attiogbé

Janvier 2025



Contenu prévisionnel

Plan de la suite (adaptation selon contraintes)

- 1 Automates
- 2 Grammaires régulières et langages réguliers
- 3 Expressions régulières
- 4 Automates finis non-déterministes et déterministes
 - Automates finis déterministes
 - Automates finis non-déterministes
- 5 Théorème de KLEENE
- 6 Minimisation et autres opérations sur les automates
- 7 Machines à états de MEALY
- 8 Ouvertures - Modélisation
- 9 Applications - Modélisation
- 10 Généralisation des machines à états

Grammaires régulières - Automates déterministes

L est un **langage régulier** $\Leftrightarrow$ il existe une grammaire régulière (type 3) G qui engendre L on note $L = \mathcal{L}(G)$.
Par abus, on utilisera $\mathcal{L}$ pour un langage.

Langage régulier - Automate à états finis

Tout **langage régulier** L est reconnu par un **automate à états finis déterministe**.

Grammaires régulières (rappel)

Grammaire régulière à droite

Lorsque les règles de production d'une grammaire G sont de la forme $X \rightarrow \alpha.X'$ où $\alpha \in T$ et $X \in N$ et $X' \in (N \cup T)$ alors la grammaire G est dite **régulière à droite**. ($X \rightarrow \alpha \mid \epsilon$ est autorisé).

On analyse une expression en la lisant de gauche vers la droite.

$$S \rightarrow \alpha.N$$

$$N \rightarrow \beta.N \mid \gamma.N \mid \eta.N \mid \dots$$

Exemple (avec une des grammaires précédentes) :

do.Suite $\Rightarrow$ *do.mi.Suite* $\Rightarrow$ *do.mi.fa.Suite* $\dots$

Grammaires régulières (rappel)

Grammaire régulière à gauche

Lorsque les règles de G sont de la forme $X \rightarrow X'.\alpha$ où $\alpha \in T$ la grammaire et $X' \in (N \cup T)$ est dite **régulière à gauche**.

On analyse une expression en la lisant de droite vers la gauche.

$$S \rightarrow N.\alpha$$

$$N \rightarrow N.\beta \mid N.\gamma \mid N.\eta \mid \dots$$

$$\text{Suite.re} \Rightarrow \text{Suite.fa.re} \Rightarrow \text{Suite.mi.fa.re} \dots \Rightarrow \text{do.mi.fa.re}$$

On peut effectuer l'analyse en partant de l'axiome de la grammaire vers le mot à reconnaître, ou du mot à reconnaître vers l'axiome.

Expressions régulières

Expressions régulières

Expression régulière

Une **expression régulière** E , sur un alphabet $\mathcal{A}$ est définie inductivement comme suit (avec les **3 opérateurs** $\cdot$ $+$ $*$) :

- $E = \epsilon$ **ou**
- $E = \alpha$ si $\alpha \in \mathcal{A}$ **ou**
- $E = E_1 + E_2$ avec E_1 et E_2 deux expressions régulières sur $\mathcal{A}$ **ou**
- $E = E_1.E_2$ avec E_1 et E_2 deux expressions régulières sur $\mathcal{A}$ **ou**
- $E = E_k^*$ avec E_k une expression régulière sur $\mathcal{A}$ **ou**
- $E = (E_k)$ avec E_k une expression régulière sur $\mathcal{A}$

C'est un **mot spécifique** construit avec les trois **opérations régulières** : concaténation, union finie, itération.

Expressions régulières : exemples

Exemple 1

Expressions régulières sur $\mathcal{A}_1 = \{a, b\}$:

$a.a$ $aaaaaaa$
 $a+a$ $a \mid a$ a^* $(a+b)$
 $a.a.a.a.a$ $aaaaabbbaaaaaaaa$
 $(a+b).aaaa$ $a+a^*$

Exemple 2

Expressions régulières suivantes sur $\mathcal{A}_2 = \{li, lo, ok, ko, c1, c2, c4, pa\}$:

$lo.li.ok.pa.ko.c1$
 $li.pa.ko.pa.ok.c1.lo$

Expressions régulières : exemples

$a.((a.b)^*.c.b^*)^*$ $a((ab)^*cb^*)^*$

$a.((a + b)^*.c.b^*)^*$ $a.((a \mid b)^*.c.b^*)^*$

$(a.(a.b + b.a)^*.c)^+$

Abstractions des structures de contrôle (en programmation) :

$+$ ou $\mid$: if ...then... else...

$*$: while

$.$: séquence

Expressions régulières = langages réguliers

Equivalence entre expressions régulières et langages réguliers :

Théorème

Toute expression régulière E_r décrit un langage régulier L_r :

$$E_r \implies L_r$$

Théorème

Tout langage régulier L_r peut être décrit par une expression régulière E_r :

$$L_r \implies E_r$$

Automates à états : concepts de base

Un **automate** est une structure mathématique fondamentale, utilisée pour modéliser, raisonner, analyser, simuler : c'est un modèle

Etat initial (E_i), **état final** (E_f), **transition** (étiquetée avec α) entre états E_2 et E_3



Automates à états

Automates à états

Un **automate à états** est défini par un n-uplet : $\langle S, A, \delta, S_0, S_f \rangle$

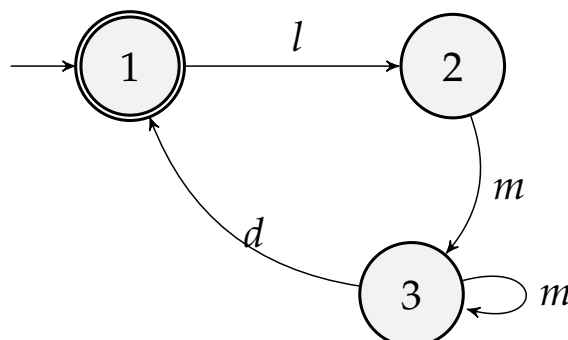
- S : ensemble d'**états** fini : $S = \{S_0, \dots, \dots\}$
- A : alphabet d'**actions** (ou **étiquettes** ou **symboles**)
- δ relation de **transition** définie de $(S \times A)$ sur S comme suit : $\delta : (S \times A) \leftrightarrow S$
- S_0 un **état initial** (avec $S_0 \in S$)
- S_f un ou des **états finaux** (c'est à dire $S_f \subseteq S$)

Automates finis déterministes (AFD ou DFA)

Automate fini déterministe

L'automate est **déterministe** lorsque il y a **un seul état initial** et la relation de transition δ **est une fonction** (et non une relation).

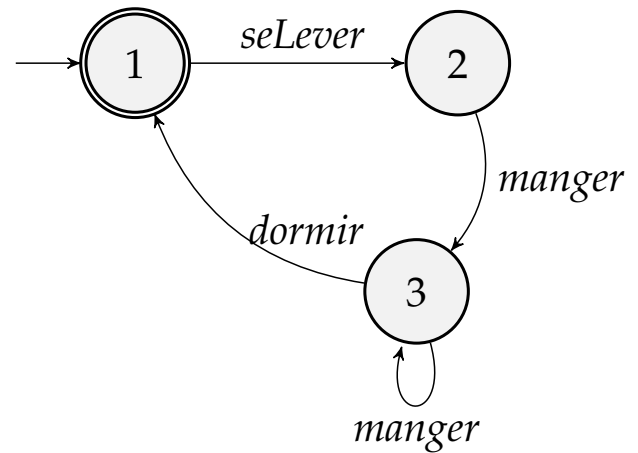
Exemple d'automate déterministe (A_0)



Automates finis déterministes

L'automate A_1 est décrit formellement par :

$S = \{1, 2, 3\}$
 $A = \{seLever, dormir, manger\}$
 $\delta : (S \times A) \leftrightarrow S$ et
 $\delta = \{ ((1, seLever), 2)$
 $, ((2, manger), 3)$
 $, ((3, dormir), 1)$
 $, ((3, manger), 3) \}$
 $S_0 = 1$
 $S_f = \{1\}$



ici, on peut noter : $\delta : (S \times A) \rightarrow S$

Automates finis déterministes

On peut représenter la relation δ par une matrice :

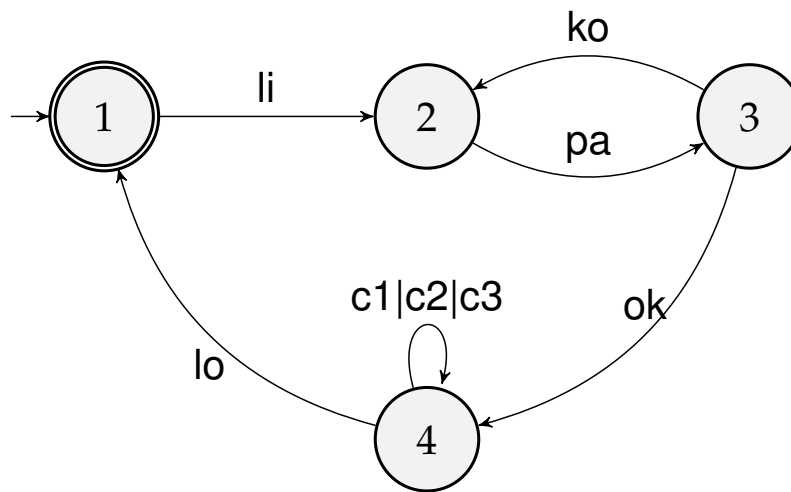
$S \setminus A$	seLever	manger	dormir
1	2		
2		3	
3		3	1

Exercice Représenter la relation par une matrice $(S \times S) \leftrightarrow A$

$S \setminus S$	1	2	3
1	-	...	-
2	-	-	...
3	...	-	...

👉 Quelle représentation choisir ? pourquoi ?

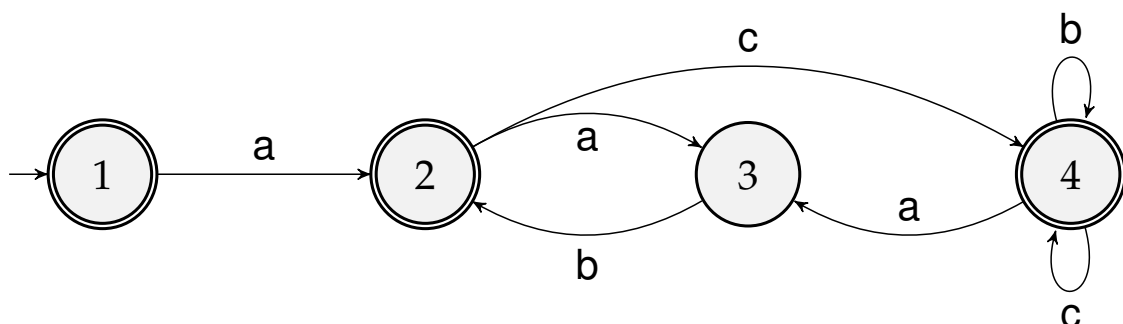
Exemple d'automate déterministe



✌ Modélisation du comportement d'un programme !

Exemple d'automates déterministes

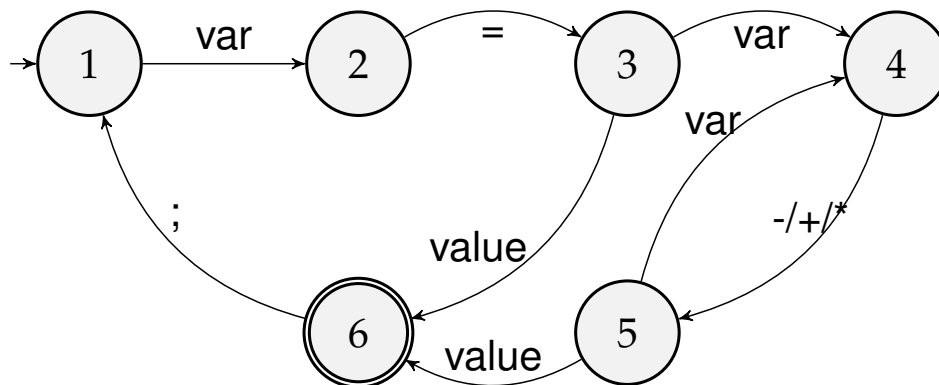
Automates qui reconnaît les mots $a((a.b)^*c.b^*)^*$ **Accepteur**



Question : Donnez quelques **mots acceptés** par cet automate

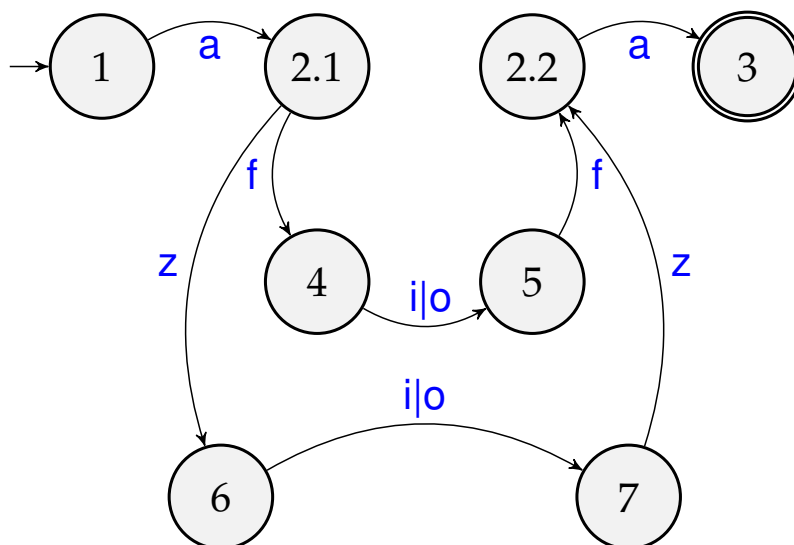
De la même façon, on peut reconnaître/analyser un programme

Exemple d'automate déterministe



Automate qui reconnaît une suite d'instructions : **Accepteur**

Exemple d'automate déterministe



$$A = \{a, i, o, f, z\}$$

$$S \rightarrow a N a$$

$$N \rightarrow f I f$$

$$| z I z$$

$$I \rightarrow i | o$$

Automate qui reconnaît/accepte **aziza, afofa, afifa, ...**

Algorithme : comportement d'un AFD accepteur

```

Etatc  $\leftarrow S_0$  {partons de l'état initial}
{On lit une suite/chaîne de caractères en entrée}
{On prendra carCourant comme le caractère suivant dans la chaîne}
while Etatc  $\neq S_f \wedge$  Etatc  $\neq$  EtatNonDef do
    lire carCourant {on parcourt la chaîne de caractères en entrée}
    afficher (Etatc, carCourant)
    Etatc  $\leftarrow \delta(\text{Etatc}, \text{carCourant})$ 
end while
if Etatc =  $S_f$  then
    Bien terminé : la suite de caractères en entrée est reconnue
else {On s'est arrêté sur un état non défini}
    Echec : la suite de caractères n'est pas reconnue
end if
{Attention : conditions + algo à parfaire + 5 constituants }
  
```

Automates finis non-déterministes (AFND)

Automate fini non-déterministe

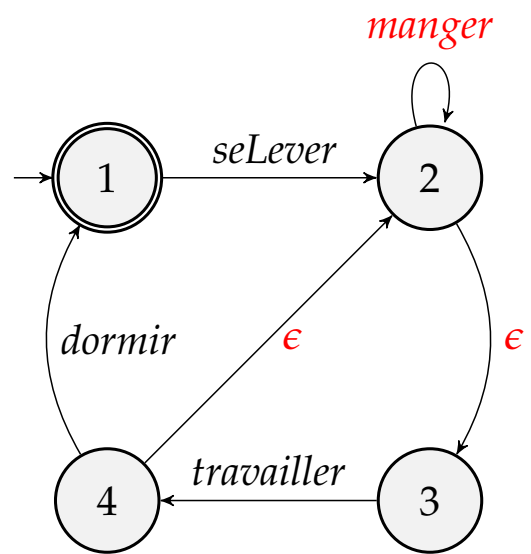
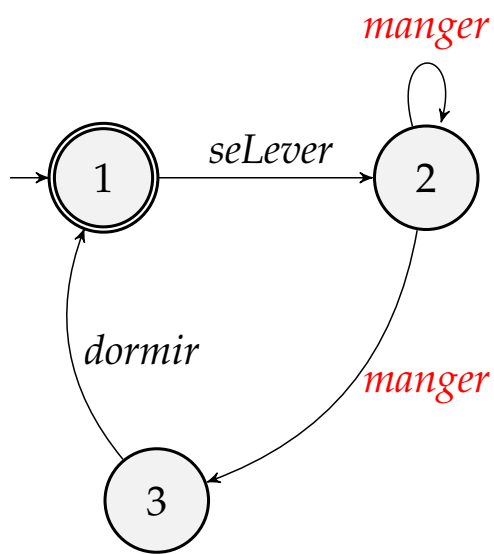
L'automate est **non-déterministe** lorsque :

- il a plusieurs **états initiaux** ou
- la relation de transition δ est bien une **relation** (et non une fonction) - c'est à dire, partant d'un état et d'un symbole, il y a plusieurs états suivants.

Une transition sans étiquette de l'alphabet est implicitement étiquetée par le mot vide ϵ : il s'agit d'une **ϵ -transition**.

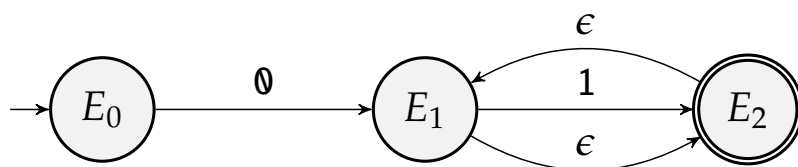
Un automate à **une/des ϵ -transitions** est un **automate non déterministe**.

Exemples d'automates non-déterministes



Automates non-déterministes

Exemple d'un Automate ND pour reconnaître : 01^*



Théorème de KLEENE

Théorème 1 : Tout langage accepté par un automate fini est régulier.

Théorème 2 : Tout langage régulier est accepté par un automate fini.

Le théorème s'énonce comme suit :

Théorème de KLEEN

Soit $\mathcal{A}$ un alphabet, pour tout langage L sur $\mathcal{A}^*$, (ou encore pour $L \subseteq \mathcal{A}^*$), les **deux assertions suivantes sont équivalentes** :

- L est un langage régulier,
- il existe un automate fini déterministe A qui reconnaît L , c'est à dire $L = \mathcal{L}(A)$

Opérations sur les automates

- Minimisation d'automate : algorithme
- Produit d'automates : algorithme
- Déterminisation AFND $\rightarrow$ AFD : algorithme
- AFD $\rightarrow$ AFDN : l'algorithme de THOMSON permet de construire un AFDN (avec des ϵ -transition) à partir d'une expression régulière.

Quelques références :

- Quelques algos www.irif.fr/~asarin/al2k2/algos.pdf
- Automate fini minimal deptinfo.unice.fr/~julia/AL/03a1.pdf
- Les automates gallium.inria.fr/~maranget/X/421/poly/automate.html
- Automates http://www-verimag.imag.fr/~perin/enseignement/RICM3/infaeg/cours/html/INF03_A&G_AEF.html

Machine à états de Mealy

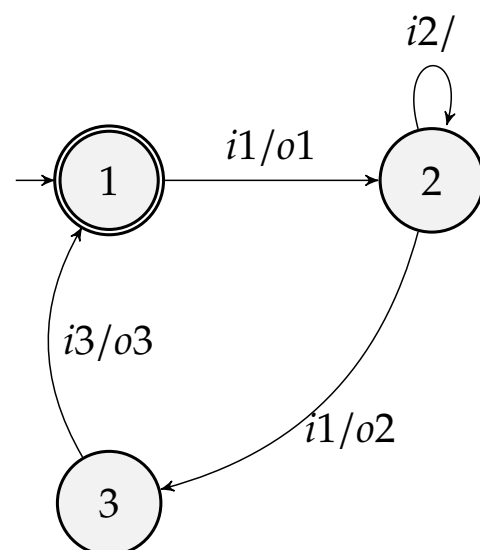
Un automate de Mealy est défini par un n-uplet $\langle S, A = In \cup Out, \delta, \delta_o, S_0, S_f \rangle$

- Ensemble d'états : S
- Ensemble d'entrées : In
(événements d'entrées, conditions)
- Ensemble de sorties : Out
(actions de sortie, traitements)
- Un état initial : S_0
- **Deux relations(fonctions) :**
 - transition $\delta : S \times In \rightarrow S$
 - **sortie** $\delta_o : S \times In \rightarrow Out$
- Etat final S_f

Machine à états de Mealy

Un automate de Mealy est défini par un n-uplet $\langle S, A = In \cup Out, \delta, \delta_o, S_0, S_f \rangle$

- Ensemble d'états : S
- Ensemble d'entrées : In
(événements d'entrées, conditions)
- Ensemble de sorties : Out
(actions de sortie, traitements)
- Un état initial : S_0
- **Deux relations(fonctions) :**
 - transition $\delta : S \times In \rightarrow S$
 - **sortie** $\delta_o : S \times In \rightarrow Out$
- Etat final S_f



Machines de Mealy : notation

- Notation des transitions : $S_s \xrightarrow{i/o} S_t$
- si l' **entrée** i est reçue alors que le système est dans l'état S_s , la **sortie** o est produite et le nouvel état du système est S_t
- i est aussi appelé le déclencheur (*trigger*).

La **notation des transitions est étendue** de la façon suivante : les transitions entre états ont la forme **evt [garde] / actions***.

evt est l'entrée reçue/lue.

[garde] : garde est une condition sur l'état du système après l'événement evt.

action* est une suite de 0 ou plusieurs actions.

Exemple d'automate de Mealy

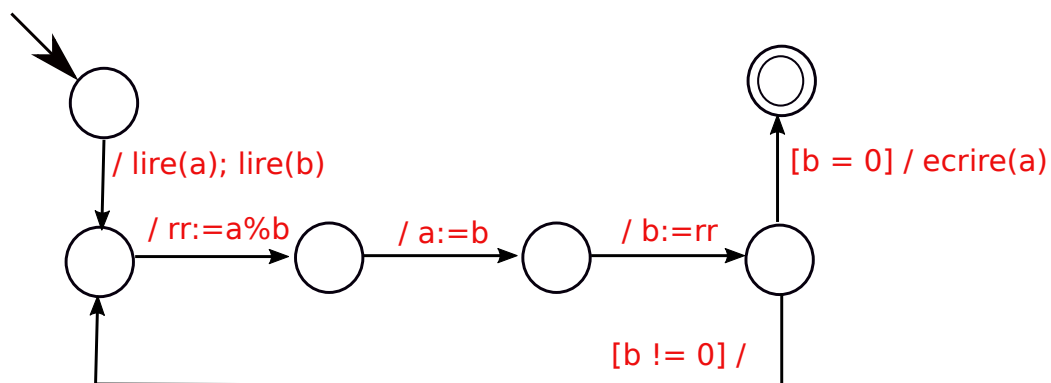


Figure: Exemple de modèle

Caractéristiques d'une machine de Mealy

- **non hiérarchique**
- un état dénote l'état complet du système
- le système est dans **un seul état à la fois**
- une **transition est atomique** ; elle ne peut être décomposée.

Terminologie

- **Espace d'états** : ensemble des états (possibles) d'un système
- **Relation de transition, Fonction de transition**
- Automate (*Automaton, Automata*)
- Machine à états finis *Finite State Machine*)
- **Graphe d'états**
- Diagramme Etat/Transition (*State/Transition Diagram*)
- Système de transitions (étiquetées) *Labelled Transition System*
- Espace d'états (*state space*)

Modélisation de systèmes

En modélisation et en construction de logiciels en particulier, on **prévoit le comportement des logiciels**, à l'aide de **modèles**. Les automates (avec les graphes) sont largement utilisés comme modèles du logiciel.

Le **comportement d'un système** (quel qu'il soit), peut être vu comme une suite (transitions) des différents états du système.

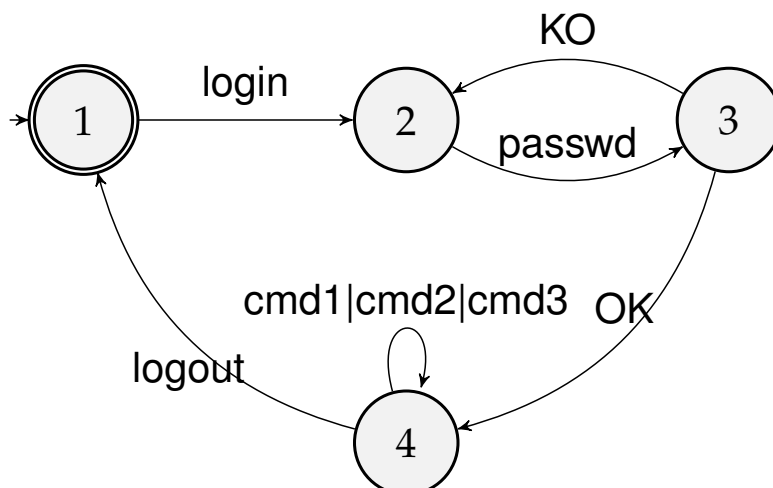
Un modèle permet de **prédire les comportements corrects et incorrects** (*bugs*) ; il faut pour cela que les modèles soient (mathématiquement) les plus précis possibles.

Modélisation de systèmes

Modélisation de système

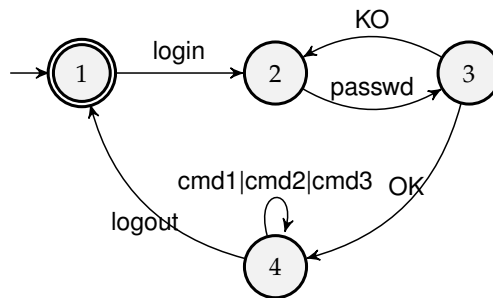
Un modèle (un automate à états) simule mathématiquement un système ; il permet d'en simuler le comportement.

Exemple Modèle d'un processus client qui se connecte à serveur.



Modélisation de systèmes

Modèle d'un **processus client (C)** qui se connecte à serveur.



mot équivalent :

$$C = (login.P)^*$$

$$P = passwd.Suite$$

$$Suite = KO.P + OK.S_1$$

$$S_1 = cmd1.S_1 + cmd2.S_1 + cmd3.S_1 + logout$$

ou bien :

$$C = (login.passw.(KO.passwd)^*.OK.(cmd1 + cmd2 + cmd3)^*.logout)^*$$

Navigation icons: back, forward, search, etc.

Modélisation de systèmes

Modélisation de systèmes (Client/serveur)

Un modèle (un automate à états) qui simule mathématiquement le **comportement d'un client**.

Un modèle (un automate à états) qui simule mathématiquement le **comportement d'un serveur**.

Interaction entre les deux : produit synchrone/asynchrone

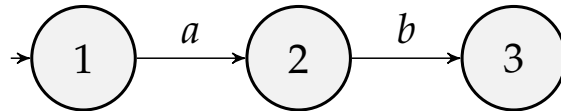
Exemple Modèle d'un processus client qui se connecte à serveur.

Comportement de l'interaction entre client(s) et serveur(s)

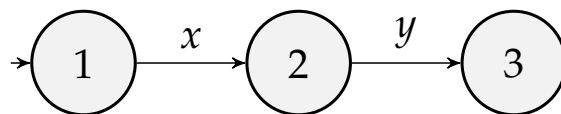
Navigation icons: back, forward, search, etc.

Modélisation de systèmes

Description du comportement d'un processus ($P1=a.b$):



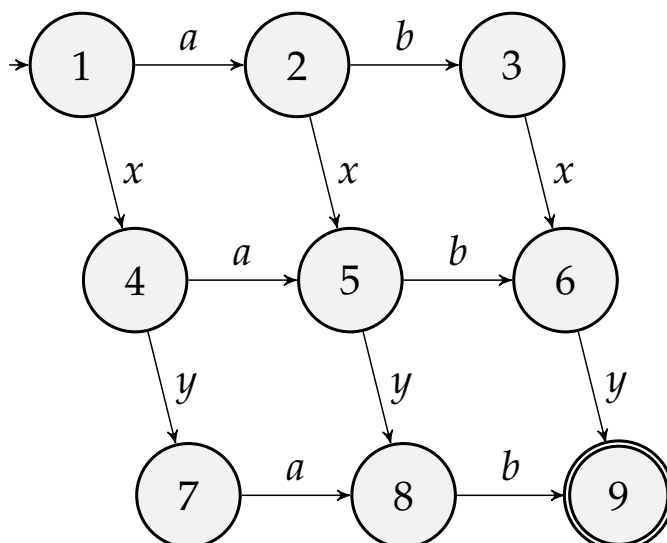
Description du comportement d'un processus ($P2=x.y$):



Comment se comporte $P1 \parallel P2$, exécution parallèle ?

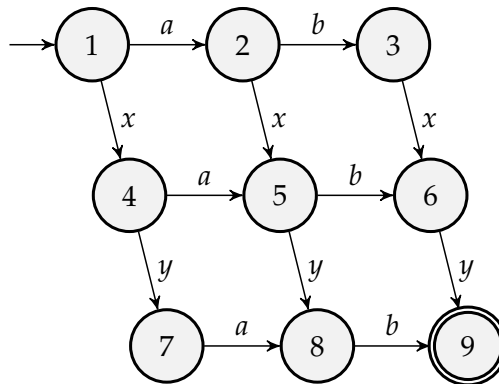
Modélisation de systèmes

Description du parallélisme entre processus ($a.b \parallel x.y$):



Etude ou analyse des systèmes (logiciels ou pas)

Traces de :



Tous les mots : axby, abxy, axyb, xayb, xyab, xaby, xyab :
sémantique comportementale

Etude ou analyse des systèmes (logiciels ou non)

Etude (analyse) des propriétés d'un logiciel/système : avec les automates, les **traces** ; **behavioural semantics**,...

Exemples

- Détection d'anomalie : mot préfixe, mot suffixe.
- Est-ce que toutes les personnes dans un bâtiment ont badgé ?
- Génération des Tests des logiciels ou des systèmes.

Extensions/généralisation des automates

Il y a **plusieurs modèles et familles de modèles à états**.

Plusieurs langages/formalismes de modélisation utilisent ce modèle.

- **System Design Language (SDL)** (normalisé par la *International Telecommunication Union* (ITU), 1988, très utilisé en Télécoms et ailleurs)
- **Statecharts** de David Harel (hiérarchiques) 1987, utilisé pas exemple dans papyrus, ...
- **Réseaux de Petri** de Carl Adam Petri (1962, 1969)
- **Algèbres de processus**, telles que *Calculus of Communicating Systems* (CCS, 1980) de R. Milner, *Communicating Sequential Processes* (CSP, 1978) de T. Hoare. C'est la famille la plus expressive des modèles à états.

Extensions/généralisation des automates

Les états et les transitions des automates peuvent être étendus pour gagner en expressivité

- **Automates à états temporisés**
(les transitions comportent des contraintes de temps)
- **Automates à états probabilistes**
(les transitions comportent des données de probabilités)
- ...

Conclusion

Mots et Langages : concepts fondamentaux.

Langages et Grammaires : formalismes de construction

Langages et Automates : description, modélisation, raisonnement

Automates : modèle fondamental en Informatique avec de nombreux algorithmes et outils

👉 Nous avons fait une introduction ; reste à approfondir

Quelques références

de nombreuses ressources (livres, cours, sites web) .

- John E. Hopcroft, Rajeev Motwani et Jeffrey D. Ullman,
Introduction to Automata Theory, Languages, and Computation,
Addison Wesley, 2001, 2e éd..

- Jacques Stern

Fondements mathématiques de l'informatique

Ediscience international, 2003 (Chapitre 1)

- Yliès Falcone, Jean-Claude Fernandez

*Automates à états finis et langages réguliers Rappels des notions
essentiels et plus de 170 exercices corrigés* Dunod, 2020

Quelques références

- Compilateurs : Principes, techniques et outils
Alfred AHO, Ravi SETHI et Jeffrey ULLMAN.
Dunod, 2000. (vers anglaise *Compilers*, Addison Wesley, 1986)
- Alfred Aho, Monica Lam, Ravi Sethi et Jeffrey Ullman
Compilateurs : principes, techniques et outils : Avec plus de 200 exercices, Pearson, 2007
- Peter Linz,
An Introduction to Formal Languages and Automata,
Jones & Bartlett Learning, 2001, 410p.

Quelques références

et des polys de cours

- Anca Muscholl,
<https://www.labri.fr/perso/anca/Langages/cours/>, Université de Bordeaux
- Gérard Duchamp, Institut Galilée
https://lipn.univ-paris13.fr/~duchamp/Enseignement/L3_Info/Supports/Cours/THL_2.pdf
- Michel Rigo, Théorie des automates et langages formels
http://www.discmath.ulg.ac.be/cours/main_autom.pdf
- Notes de révision : Automates et langages
Benjamin MONMEGE et Sylvain SCHMITZ
http://www.lsv.fr/~schmitz/teach/2013_agreg/notes-r62.pdf
- Notes de cours, de Christine Solnon
<http://perso.citi-lab.fr/csolnon/langages.pdf>

Travaux pratiques

En TD, construction de plusieurs automates ; serviront en entrée d'une application à construire.

Le principe est donné dans la figure 2 ; implantation en kotlin.

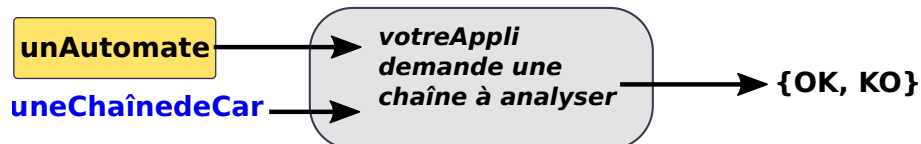


Figure: Architecture simplifiée de l'application à écrire

Travaux pratiques : menu à proposer

---- Nantes Université, BUT2 Informatique, 2024 ----

-- TP - Modélisation et analyse à l'aide des automates --

--- Menu de l'application (développée par par Moa TOA) ---

1. Smiley (pour reconnaître un des smileys)
2. HH:MM (pour reconnaître une heure bien formée)
3. JJ/MM/AAAA
4. Adresses électroniques
5. Polynômes
6. Expressions arithmétiques
7. Polynômes

...

99. Arrêt de l'application

Votre choix (1-99) ?

Je vous demanderai ensuite la chaîne à analyser, Merci

TP : encodage des automates déterministes

Encodage **un automate** A : **une classe avec le quintuplet** qui définit formellement l'automate ; (en plus du nom de l'automate).

Il faudra des *getter/setter* pour chaque constituant.

Au moins 3 classes :

- Etat
- Symbole
- Automate

TP : encodage des transitions

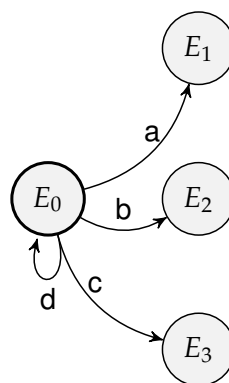


Figure: Un noeud d'un automate ...

Exemple : l'état E_0 aura comme attribut la **hashmap** outgoingTransition suivante : $\{\langle d, E_0 \rangle, \langle a, E_1 \rangle, \langle b, E_2 \rangle, \langle c, E_3 \rangle\}$

TP : encodage des automates déterministes

- Chaque état (une classe) a parmi ses attributs les transitions sortantes et quel/s symbole/s de l'alphabet permettent d'atteindre ses transitions.
- Les transitions sortantes sont par exemple représentées par une `HashMap<clé, valeur>`, où la clé est un symbole et la valeur l'état atteint.
- Ainsi dans chaque état, on définit facilement une fonction qui décide si un symbole s lu est une action sortante ou pas (s est parmi les clés) ; et lorsque c'est le cas, une fonction qui donne immédiatement l'état atteint : `outgoingTransition(ss)`.